ボケの全件のアノテーションに向けた検証と最適化

1. 概要

約 9,250 万件のボケに発話の型・画像外の知識・制約の順守のラベルを付ける処理を，Bokete_Annotation_programs_v5.0 にボケの段階として組み込んだ（v5.2.0）．全件の実行の前に，検証と最適化を行った．判定器は，確認のうえ Qwen3-VL-4B 単独（ex016 の二段階の微調整）とした．全件の実行は次の指示で行う．

結論は次のとおりである．

項目	結果
精度の再現	パッケージのコードで，検証データの精度が再現された（最終評価用：6 区分 0.711，画像外の知識 0.694．研究時は 0.711，0.690）
高速な推論の正しさ	実データの無作為の 3,000 件で，通常の推論との判定の一致は発話の型 99.7%，画像外の知識 99.6%
再現性	同じ設定での再実行で判定は全て一致した（確率の完全一致は 99.8% の行）
制約の順守	付け替えた組との判別で AUC 0.812（研究時 0.81），カルタの規則の正解率 0.97（研究時 0.97）
例外的な入力	画像がない，空のボケ，ボケの多いお題（344 件），形式がないお題で止まらずに処理できた
速度	定常で約 345 件/秒（ex017 の約 128 件/秒の約 2.7 倍）．全件で約 3.1 日（ボケの段階の合計で約 3.2 日）
中断と再開	途中で止めて同じコマンドで再実行すると，未処理のシャードだけを処理した
ディスク	ボケの段階の出力は約 7GB

2. 付与するラベル

`boke_annotation/part_XXXX.parquet`（1 行が 1 件のボケ）に，次の列を付ける．詳細はパッケージの `docs/data_format.md` の 2.4 節にある．

列	内容	方法
`voice6`，`p_voice_*`	発話の型 6 区分（信頼性が低い）と確率	分類器（Qwen3-VL-4B + LoRA）
`voice3`	発話の型 3 区分（`character`，`outside`，`answer`）	`voice6` から導く
`outside`，`p_outside`	画像外の知識と確率	分類器
`karuta_card`，`karuta_complies`	カルタの札のかなと順守	規則（書き出しの音）
`compliance_score`	穴埋め・お題文の順守の得点	同じお題の他のボケとの類似度（ruri-v3-310m）
`uses_image_text`	画像中の文字への依存	形式による規則
`predicted`，`empty_text`	予測したか，本文が空か	—

定義書はパッケージの `docs/boke_label_definition.md` である．研究用の第 3 版に，次の規則を追記した．これは，検証データの見直しで定めて目視のラベルに適用していたものである．- 形式が通常やカルタでも，お題画像が空欄・空の吹き出し・問いを課していれば回答とする．

3. データの確認

項目	値
ボケの件数	92,495,946 件（814 ファイル．1 ファイル約 11 万件．ボケ ID は一意）
お題の数	6,580,989（1 お題あたり平均 14.1 件，中央値 9 件，最大 1,000 件．200 件を超えるお題は 12,502）
形式ごとの件数	通常 4,700 万，穴埋め 3,304 万，お題文 1,099 万，カルタ 146 万
お題のラベルがない（画像がない）お題	452 お題，3,133 件（予測しない）
本文が空のボケ	16,903 件（予測し，`empty_text` で示す）
本文の長さ	平均 14.8 字，99% 点 54 字，最大 87 字

ボケの表は，お題 ID の範囲ごとのファイルに分かれている．そのため，1 ファイルを 1 シャードとすると，同じお題のボケがシャードをまたがない．

4. 最適化

4.1 構成

- **一覧（boke_list）**：ボケの表の 1 ファイルごとに，ボケとお題のラベル（形式，代表の画像）を結合する．（代表の画像，形式，ボケ ID）の順に並べて保存する．同じお題画像のボケが連続する．
- **推論（boke_predict）**：
  - 全ての未処理のシャードを 1 つのデータローダーで順に処理する．シャードごとにワーカーを起動し直すと，起動（約 2.5 分）が 814 回かかるためである．
  - 1 つのシャードの全てのまとまりが終わるごとに保存する．
  - 1 つのまとまり（チャンク）は，同じ代表の画像・形式の連続するボケ（最大 256 件）である．
  - ボケが約 512 件になるようにチャンクをまとめて推論する．

4.2 GPU の処理の最適化

ex017 のプレフィックスのキャッシュ（Qwen3-VL-4B，約 128 件/秒）を基に，次を行った．

改良	内容	効果（GPU のみの速度）
LoRA の統合	推論の前に LoRA を重みに統合する（`merge_and_unload`）	147 件/秒 → 197 件/秒
ボケの部分のまとめ方	ボケの部分を長さで並べ替え，128 件ずつ計算する．キャッシュは 128 件分だけ複製する	詰め物の割合 68% → 小．197 件/秒 → 約 360 件/秒

GPU の時間の内訳（ボケ 20,024 件，55 秒）は次のとおりであり，多数の小さな演算の起動が残りの主な費用である．

部分	秒数	割合
ボケの部分	約 30	54%
画像の符号化	7.4	13%
位置の計算（`get_rope_index`）	2.4	4%
プレフィックスの言語モデルなど	残り	

プレフィックスの詰め物（20%）を減らす分割や，`torch.compile`・CUDA グラフも考えたが，見込める効果に対して変更の危険が大きいため見送った．

4.3 設定ごとの速度（2 シャード，278,480 件）

設定	定常の速度	起動の時間	全体の速度
前処理 8 プロセス，512 件ずつ，128 件ずつ（**採用**）	344 件/秒	156 秒	290 件/秒
前処理 16 プロセス，512 件ずつ	348 件/秒	297 秒	255 件/秒
前処理 16 プロセス，1,024 件ずつ	348 件/秒	288 秒	257 件/秒

- 定常の速度はどの設定でもほぼ同じであり，GPU の計算が律速である．
- 前処理のプロセスを増やすと起動が長くなるだけであった．
- 起動が長いのは，仮想環境が Windows のドライブ（/mnt/d）にあり，各プロセスでの torch・transformers の読み込みが遅いためと考えられる．全件の実行では 1 回だけかかる．

5. 検証

検証	方法	結果
精度の再現（verify_eval）	パッケージのコードで検証データ 620 件を予測し，研究時（ex016）の予測と合意ラベルでの F1 を比べる	判定の一致：発話の型 99.8%，画像外の知識 99.7%．最終評価用 F1：6 区分 0.711（研究時 0.711），画像外の知識 0.694（0.690）．調整用：0.759（0.758），0.667（0.662）
同じ処理での再現性	同じプロセスで 2 回予測する	確率まで完全に一致
実データでの一致（verify_sample）	予測済みの 2 シャードから無作為に 3,000 件を選び，通常の推論（LoRA を統合しない 1 件ごと）と比べる	判定の一致：発話の型 99.7%，画像外の知識 99.6%．画像外の知識の確率の差は平均 0.003，99% 点 0.04，最大 0.10
再実行の一致	同じ設定で，別のプロセスで同じシャードを予測する	判定は 100% 一致．確率は 99.8% の行で完全に一致（最大の差 0.095）
設定の違い	まとめる件数を 1,024 にする	判定の一致 99.8%（bfloat16 の計算の順序の違い）．全件を同じ設定で処理すること

検証	方法	結果
制約の順守（verify_compliance）	パッケージの得点の関数で，付け替えた組との判別を研究時と同じ条件で評価する	AUC 0.812（穴埋め 0.739，お題文 0.882）．カルタの規則の正解率 0.971（102 件）
例外的な入力（verify_edge）	画像がないお題，空のボケ，ボケが 344 件のお題，形式がないお題を含む一覧を処理する	全 350 件を出力．画像がないボケは予測なし（predicted 偽）．空のボケは予測して empty_text 真．344 件のお題は分割して全て予測
中断と再開	3 シャード目の途中（45/240）で止め，同じコマンドで再実行する	処理済みの 2 シャードは飛ばし，3 シャード目だけを最初から処理した．途中の結果のファイルは残らない
テスト	パッケージの pytest	20 件（ボケの段階の 14 件を追加）が全て成功

先頭の 2 シャード（245,828 件）の予測の分布は次のとおりであり，検証データの分布とおおむね一致した．- 6 区分：セリフ 43%，回答 20%，名付け 18%，状況の説明 7%，ツッコミ 7%，心の声 5%．- 画像外の知識：32%（検証データは 22〜25%）．- カルタの順守：94%．

6. 全件の実行の見積もり

段階	内容	時間
boke_list	814 シャードの一覧の作成	数分
boke_predict	発話の型と画像外の知識（約 345 件/秒）	約 3.1 日
boke_compliance	穴埋め・お題文の約 4,400 万件の文の埋め込み（約 6,600 件/秒）とカルタの規則	約 2 時間
boke_finalize	結合と集計	数分

実行のコマンドは次のとおりである（パッケージのディレクトリで）．

```
../.venv_annotate/bin/python annotate.py --db_dir
 ↪ /mnt/d/Bokete_Scraping_v3.2/outputs/BoketeDBv4 \
   --image_dir /mnt/d/Bokete_Scraping_v3.2/datasets/images \
   --stages boke_list boke_predict boke_compliance boke_finalize
```

- 必要な資源は次のとおりである．
  - GPU のメモリ：最大約 24GB（確保量）．
  - ディスク：約 7GB．
  - モデルの取得：Qwen3-VL-4B 約 9GB，ruri-v3-310m 約 1GB．
- 約 3 日の連続実行になる．実行中は GPU を他の処理に使わないこと．
- 停電や再起動に備え，中断しても同じコマンドで再開できる（シャード単位で保存）．
- 出力先は outputs/BoketeDBv5_annotation/（お題画像の annotation.parquet と同じ場所）である．
- 動作確認の出力（outputs/test_boke/）は，全件の出力とは別のディレクトリにある．

7. 未解決の論点

1. **6 区分の信頼性**：6 区分は信頼性が低い（F1 約 0.71）．README と定義書に明記した．
2. **画像外の知識の誤り**：引用の見逃しと，画像中の作品の登場人物の誤検出が残る（F1 約 0.69）．
3. **さらなる高速化**：次の候補があるが，今回は見送った．
   - 位置の計算のベクトル化（約 4%）．
   - プレフィックスの長さによる分割（約 2%）．
   - `torch.compile`・CUDA グラフ（効果は未計測）．
4. **パッケージの公開**：パッケージは独立した Git のリポジトリであり，変更はまだ commit していない．重み（161MB）は GitHub の容量の制限を超えるため，Releases などで配布する必要がある．

付録：作成・変更したファイル

- パッケージ（Bokete_Annotation_programs_v5.0，v5.2.0）
  - 追 加：`bokete_annotation/boke.py`， `boke_rules.py`， `boke_pipeline.py`， `tests/test_boke.py`，`docs/boke_label_definition.md`，`models/boke_qwen3vl4b_lora.pt`
  - 変更：`annotate.py`，`config.json`（`boke` の項目），`requirements.txt`（peft，pykakasi，sentencepiece，protobuf），`README.md` ／ `README.pdf`，`docs/data_format.md`，`docs/method.md`（6 章），`models/README.md`，`models/checksums.sha256`，`bokete_annotation/__init__.py`
- 研究用のリポジトリ
  - `programs/ex019_boke_full/`（verify_eval, verify_sample, verify_compliance, verify_edge, tune_speed, profile_predict）
  - `outputs/ex019_boke_full/`